

1.Escribir un programa que calcule la media de los elementos de un vector.

Pseudocódigo:

Algoritmo MediaVector

Definir n, i Como Entero

Definir suma, media Como Real

Dimension v[100]

Escribir "Ingrese n:"

Leer n

suma <- 0

Para i <- 1 Hasta n Hacer

Escribir "v[, i, "]:"

Leer v[i]

suma <- suma + v[i]

FinPara

media <- suma / n

Escribir "Media: ", media

FinAlgoritmo

Prueba de Escritorio:

Valores de prueba: n = 3, vector v = {4, 5, 6}

Paso	i	v[i]	suma	media
Inicial	-	-	0	-
1	1	4	4	-
2	2	5	9	-

3	3	6	15	-
Final	-	-	15	5

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int n;
    float v[100], suma = 0, media;

    cout << "Ingrese el tamaño del vector: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Elemento " << i << ": ";
        cin >> v[i];
        suma += v[i];
    }

    media = suma / n;
    cout << "La media es: " << media << endl;

    return 0;
}
```

2. Escribir un programa que calcule y devuelva la suma al cuadrado de las componentes de un vector

Pseudocódigo:

Algoritmo SumaCuadrados

```

Definir n, i Como Entero
Definir suma Como Real
Dimension v[100]

Escribir "Ingrese n:"
Leer n

suma <- 0
Para i <- 1 Hasta n Hacer
    Escribir "v[, i, "]:"
    Leer v[i]
    suma <- suma + (v[i] * v[i])
FinPara

Escribir "Suma de cuadrados: ", suma
FinAlgoritmo

```

Prueba de Escritorio:

Valores de prueba: $n = 2$, vector $v = \{3, 4\}$

Paso	i	v[i]	Operación	suma
Inicial	-	-	-	0
1	1	3	$0 + (3 \times 3)$	9
2	2	4	$9 + (4 \times 4)$	25

Código (C++):

```

#include <iostream>

using namespace std;

```

```

int main() {
    int n;
    float v[100], suma = 0;

    cout << "Ingrese el tamaño del vector: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Elemento " << i << ": ";
        cin >> v[i];
        suma += (v[i] * v[i]);
    }

    cout << "La suma al cuadrado es: " << suma << endl;

    return 0;
}

```

3. Escribir un programa que calcule el producto escalar de dos vectores. Si v y w son los vectores y n su tamaño, el producto escalar se calcula como la sumatoria de $V(i) \cdot W(i)$

Pseudocódigo:

Algoritmo ProductoEscalar

Definir n, i Como Entero

Definir suma Como Real

Dimension $v[100], w[100]$

Escribir "Ingrese n:"

Leer n

Para $i \leftarrow 1$ Hasta n Hacer

Leer $v[i]$

```
FinPara
```

```
Para i <- 1 Hasta n Hacer
```

```
    Leer w[i]
```

```
FinPara
```

```
suma <- 0
```

```
Para i <- 1 Hasta n Hacer
```

```
    suma <- suma + (v[i] * w[i])
```

```
FinPara
```

```
    Escribir "Producto escalar: ", suma
```

```
FinAlgoritmo
```

Prueba de Escritorio:

Valores de prueba: $n = 2$, $V = \{2, 3\}$, $W = \{4, 5\}$

Iteración i	V[i]	W[i]	V[i] * W[i]	suma (acumulado)
Inicial	-	-	-	0
1	2	4	8	8
2	3	5	15	23

Código (C++):

```
#include <iostream>

using namespace std;
```

```
int main() {
    int n;
```

```

float v[100], w[100], suma = 0;

cout << "Tamano de los vectores: ";
cin >> n;

cout << "Ingrese datos del Vector V:\n";
for (int i = 0; i < n; i++) cin >> v[i];

cout << "Ingrese datos del Vector W:\n";
for (int i = 0; i < n; i++) cin >> w[i];

for (int i = 0; i < n; i++) {
    suma += (v[i] * w[i]);
}

cout << "El producto escalar es: " << suma << endl;

return 0;
}

```

4. Escribir el programa que calcule el producto de un número por un vector (devuelve el vector resultante de multiplicar todas las componentes del vector inicial por un número)

Pseudocódigo:

Algoritmo ProductoNumeroVector

Definir n, i Como Entero

Definir k Como Real

Dimension v[100]

Escribir "Ingrese n y el numero k:"

Leer n, k

Para i <- 1 Hasta n Hacer

```

        Leer v[i]
        v[i] <- v[i] * k
    FinPara

    Para i <- 1 Hasta n Hacer
        Escribir v[i], " "
    FinPara
FinAlgoritmo

```

Prueba de Escritorio:

Valores de prueba: $n = 2$, $k = 3$, vector inicial $V = \{2, 5\}$

Iteración i	v[i] original	Operación	v[i] modificado
1	2	2×3	6
2	5	5×3	15

Código (C++):

```

#include <iostream>
using namespace std;

int main() {
    int n;
    float k, v[100];

    cout << "Tamano del vector: ";
    cin >> n;
    cout << "Ingrese el escalar (k): ";
    cin >> k;
}

```

```

    cout << "Ingrese los elementos del vector:\n";
    for (int i = 0; i < n; i++) {
        cin >> v[i];
        v[i] = v[i] * k;
    }

    cout << "Vector resultante: ";
    for (int i = 0; i < n; i++) {
        cout << v[i] << " ";
    }
    cout << endl;

    return 0;
}

```

5. Escribir un programa que calcule el vector resultante de sumar a cada una de las componentes de un vector dado, un número real

Pseudocódigo:

Algoritmo SumaEscalar

Definir n, i Como Entero

Definir k Como Real

Dimension v[100]

Escribir "Ingrese n y el numero real k a sumar:"

Leer n, k

Para i <- 1 Hasta n Hacer

 Escribir "v[, i, "]:"

 Leer v[i]

 v[i] <- v[i] + k

FinPara


```

    Escribir "Vector resultante:"
    Para i <- 1 Hasta n Hacer
        Escribir v[i], " "
    FinPara
FinAlgoritmo

```

Prueba de Escritorio:

Valores de prueba: $n = 2$, $k = 1.5$, vector inicial $V = \{10, 20\}$

Iteración i	v[i] original	Operación	v[i] final
1	10	$10 + 1.5$	11.5
2	20	$20 + 1.5$	21.5

Código (C++):

```

#include <iostream>
using namespace std;

int main() {
    int n;
    float k, v[100];

    cout << "Tamano del vector: ";
    cin >> n;
    cout << "Numero real a sumar (k): ";
    cin >> k;

    cout << "Ingrese los elementos del vector:\n";
    for (int i = 0; i < n; i++) {
        cin >> v[i];
    }
}

```

```

        v[i] += k;
    }

    cout << "Vector resultante: ";
    for (int i = 0; i < n; i++) {
        cout << v[i] << " ";
    }
    cout << endl;

    return 0;
}

```

6. Escribir un programa que devuelva el mínimo de los elementos de un vector.

Pseudocódigo:

Algoritmo MinimoVector

Definir n, i Como Entero

Definir minimo Como Real

Dimension v[100]

Escribir "Ingrese n:"

Leer n

Para i <- 1 Hasta n Hacer

Leer v[i]

FinPara

minimo <- v[1]

Para i <- 2 Hasta n Hacer

Si v[i] < minimo Entonces

minimo <- v[i]

FinSi

FinPara

Escribir "El minimo es: ", minimo

FinAlgoritmo

Prueba de Escritorio:

Valores de prueba: $n = 3$, $V = \{8, 3, 5\}$

Iteración i	v[i]	¿v[i] < minimo?	Nuevo minimo
Inicial	-	-	8 (es el v[1])
2	3	Sí ($3 < 8$)	3
3	5	No ($5 < 3$)	3

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int n;
    float v[100], minimo;

    cout << "Tamano del vector: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Elemento " << i << ": ";
        cin >> v[i];
    }
```

```

    minimo = v[0]; // Asumimos que el primero es el menor

    for (int i = 1; i < n; i++) {
        if (v[i] < minimo) {
            minimo = v[i];
        }
    }

    cout << "El valor minimo es: " << minimo << endl;

    return 0;
}

```

7.Escribir un programa que devuelva el máximo de los elementos de un vector

Pseudocódigo:

Algoritmo MaximoVector

Definir n, i Como Entero

Definir maximo Como Real

Dimension v[100]

Escribir "Ingrese n:"

Leer n

Para i <- 1 Hasta n Hacer

Leer v[i]

FinPara

maximo <- v[1]

Para i <- 2 Hasta n Hacer

Si v[i] > maximo Entonces

```

        maximo <- v[i]
    FinSi
FinPara

    Escribir "El maximo es: ", maximo
FinAlgoritmo

```

Prueba de Escritorio:

Valores de prueba: $n = 3$, $V = \{4, 9, 2\}$

Iteración i	v[i]	¿v[i] > maximo?	Nuevo maximo
Inicial	-	-	4 (es el $v[1]$)
2	9	Sí ($9 > 4$)	9
3	2	No ($2 > 9$)	9

Código (C++):

```

#include <iostream>

using namespace std;

int main() {
    int n;

    float v[100], maximo;

    cout << "Tamano del vector: ";
    cin >> n;

    for (int i = 0; i < n; i++) {

```

```

        cout << "Elemento " << i << ": ";
        cin >> v[i];
    }

    maximo = v[0];

    for (int i = 1; i < n; i++) {
        if (v[i] > maximo) {
            maximo = v[i];
        }
    }

    cout << "El valor maximo es: " << maximo << endl;

    return 0;
}

```

8.Escribir un programa que busque el menor y el mayor elemento ingresado en una matriz de $m \times n$

Pseudocódigo:

Algoritmo MinMaxMatriz

Definir m, n, i, j Como Entero

Definir minimo, maximo Como Real

Dimension matriz[20, 20]

Escribir "Ingrese filas (m) y columnas (n):"

Leer m, n

Para $i \leftarrow 1$ Hasta m Hacer

 Para $j \leftarrow 1$ Hasta n Hacer

```

        Escribir "Elemento [" , i , "]" [ , j , "]" : "
        Leer matriz[i, j]
    FinPara
FinPara

minimo <- matriz[1, 1]
maximo <- matriz[1, 1]

Para i <- 1 Hasta m Hacer
    Para j <- 1 Hasta n Hacer
        Si matriz[i, j] < minimo Entonces
            minimo <- matriz[i, j]
        FinSi
        Si matriz[i, j] > maximo Entonces
            maximo <- matriz[i, j]
        FinSi
    FinPara
FinPara

Escribir "Menor: " , minimo
Escribir "Mayor: " , maximo
FinAlgoritmo

```

Prueba de Escritorio:

Valores: matriz 2 x 2 con valores {{5, 12}, {2, 8}}

i	j	matriz[i][j]	¿< minimo?	¿> maximo?	minimo actual	maximo actual
Inicial	-	-	-	-	5	5

1	1	5	No	No	5	5
1	2	12	No	Sí	5	12
2	1	2	Sí	No	2	12
2	2	8	No	No	2	12

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int m, n;
    float matriz[20][20], minimo, maximo;

    cout << "Filas (m): ";
    cin >> m;
    cout << "Columnas (n): ";
    cin >> n;

    // Ingreso de datos
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cout << "Elemento [" << i << "][" << j << "]: ";
            cin >> matriz[i][j];
        }
    }
}
```



```

// Inicializamos con el primer elemento de la matriz
minimo = matriz[0][0];
maximo = matriz[0][0];

// Búsqueda
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (matriz[i][j] < minimo) minimo = matriz[i][j];
        if (matriz[i][j] > maximo) maximo = matriz[i][j];
    }
}

cout << "\nEl elemento menor es: " << minimo << endl;
cout << "El elemento mayor es: " << maximo << endl;

return 0;
}

```

9. Escribir un programa que sume los elementos de cada fila de una matriz de $m \times n$

Pseudocódigo:

Algoritmo SumaFilas

Definir m, n, i, j Como Entero

Definir suma Como Real

Dimension matriz[20, 20]

Escribir "Ingrese filas (m) y columnas (n):"

Leer m, n

Para $i \leftarrow 1$ Hasta m Hacer

Para $j \leftarrow 1$ Hasta n Hacer

Leer matriz[i, j]

FinPara

```
FinPara
```

```
Escribir "--- Suma por Filas ---"
```

```
Para i <- 1 Hasta m Hacer
```

```
    suma <- 0 // Se reinicia al inicio de cada fila
```

```
    Para j <- 1 Hasta n Hacer
```

```
        suma <- suma + matriz[i, j]
```

```
    FinPara
```

```
    Escribir "Fila ", i, ": ", suma
```

```
FinPara
```

```
FinAlgoritmo
```

Prueba de Escritorio:

Valores: matriz 2 x 2 con valores {{1, 2}, {3, 4}}

i (Fila)	suma (Inicio)	j	matriz[i][j]	suma (Acumulado)	Imprime
1	0	1	1	$0 + 1 = 1$	-
1	1	2	2	$1 + 2 = 3$	"Fila 1: 3"
2	0	1	3	$0 + 3 = 3$	-
2	3	2	4	$3 + 4 = 7$	"Fila 2: 7"

Código (C++):

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int m, n;
```

```

float matriz[20][20];

cout << "Filas (m): "; cin >> m;
cout << "Columnas (n): "; cin >> n;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        cout << "Elemento [" << i << "][" << j << "]: ";
        cin >> matriz[i][j];
    }
}

cout << "\nResultados:\n";
for (int i = 0; i < m; i++) {
    float suma = 0; // Se declara y reinicia aquí
    for (int j = 0; j < n; j++) {
        suma += matriz[i][j];
    }
    cout << "La suma de la fila " << i << " es: " << suma
<< endl;
}

return 0;
}

```

10. Escribir un programa que sume los elementos de cada columna de una matriz de $m \times n$

Pseudocódigo:

Algoritmo SumaColumnas

Definir m, n, i, j Como Entero

Definir suma Como Real

Dimension matriz[20, 20]

```

// (Asumimos el mismo bloque de ingreso de datos que el
ej. 9)

Leer m, n

Escribir "--- Suma por Columnas ---"

Para j <- 1 Hasta n Hacer // Ahora las columnas mandan
    suma <- 0
    Para i <- 1 Hasta m Hacer // Y recorremos las filas
        hacia abajo
            suma <- suma + matriz[i, j]
        FinPara
    Escribir "Columna ", j, ": ", suma
FinPara
FinAlgoritmo

```

Prueba de Escritorio:

Valores: matriz 2 x 2 con valores {{1, 2}, {3, 4}}

j (Col)	suma (Inicio)	i (Fila)	matriz[i][j]	suma (Acumulado)	Imprime
1	0	1	1	$0 + 1 = 1$	-
1	1	2	3	$1 + 3 = 4$	"Columna 1: 4"
2	0	1	2	$0 + 2 = 2$	-
2	2	2	4	$2 + 4 = 6$	"Columna 2: 6"

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int m, n;

    float matriz[20][20];

    cout << "Filas (m): "; cin >> m;
    cout << "Columnas (n): "; cin >> n;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matriz[i][j];
        }
    }

    // Aquí invertimos el recorrido
    for (int j = 0; j < n; j++) {
        float suma = 0;
        for (int i = 0; i < m; i++) {
            suma += matriz[i][j];
        }
        cout << "La suma de la columna " << j << " es: " <<
suma << endl;
    }

    return 0;
}
```

11.Escribir un programa que sume los elementos de la diagonal que va de izquierda a derecha en una matriz de m x n

Pseudocódigo:

Algoritmo SumaDiagonalPrincipal

Definir n, i Como Entero

Definir suma Como Real

Dimension matriz[20, 20]

Escribir "Ingrese tamaño de la matriz cuadrada (n):"

Leer n

// (Ingreso de datos)

suma <- 0

Para i <- 1 Hasta n Hacer

 suma <- suma + matriz[i, i] // Ambos índices son

iguales

FinPara

Escribir "Suma diagonal principal: ", suma

FinAlgoritmo

Prueba de Escritorio:

Matriz 3 x 3, valores diagonales: matriz[1][1] = 5, matriz[2][2] = 2, matriz[3][3] = 4

i	Coordenada	Valor	suma
1	[1][1]	5	5
2	[2][2]	2	7
3	[3][3]	4	11

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int n;
    float matriz[20][20], suma = 0;

    cout << "Tamano de la matriz cuadrada (n): "; cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matriz[i][j];
        }
    }

    // Un solo bucle para la diagonal
    for (int i = 0; i < n; i++) {
        suma += matriz[i][i];
    }

    cout << "La suma de la diagonal principal es: " << suma <<
endl;
    return 0;
}
```

12. Escribir un programa que sume los elementos de la diagonal que va de derecha a izquierda en una matriz de $m \times n$

Pseudocódigo:

Algoritmo SumaDiagonalSecundaria

Definir n , i Como Entero

```

Definir suma Como Real
Dimension matriz[20, 20]

Leer n
// (Ingreso de datos)

suma <- 0
Para i <- 1 Hasta n Hacer
    // En PSeInt los indices inician en 1, por lo que la
formula cambia a n - i + 1
    suma <- suma + matriz[i, n - i + 1]
FinPara

Escribir "Suma diagonal secundaria: ", suma
FinAlgoritmo

```

Prueba de Escritorio:

Matriz 3 x 3 (n=3). Diagonales inversas: [1][3], [2][2], [3][1] (Indices base 1). Supongamos que esos valores son 10, 20, 30.

i (Fila)	Operación Columna (3 - i + 1)	Coordenada	Valor	suma
1	$3 - 1 + 1 = 3$	[1][3]	10	10
2	$3 - 2 + 1 = 2$	[2][2]	20	30
3	$3 - 3 + 1 = 1$	[3][1]	30	60

Código (C++):

```

#include <iostream>
using namespace std;

```



```

int main() {
    int n;
    float matriz[20][20], suma = 0;

    cout << "Tamano de la matriz cuadrada (n): "; cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matriz[i][j];
        }
    }

    for (int i = 0; i < n; i++) {
        // En C++ los indices inician en 0, la formula es: n -
1 - i
        suma += matriz[i][n - 1 - i];
    }

    cout << "La suma de la diagonal secundaria es: " << suma
<< endl;
    return 0;
}

```

13. Escribir un programa que copie los elementos de los bordes en sentido horario en una matriz de m x n en un vector resultante llamado v.

Pseudocódigo:

Algoritmo BordesHorario

Definir m, n, i, j, k Como Entero

Dimension matriz[10, 10], v[100]

Escribir "Filas (m) y Columnas (n):"

```

Leer m, n
// (Asumimos el ingreso normal de la matriz aquí)

k <- 1 // Indice para el vector resultante

// 1. Fila Superior (Fila 1, todas las columnas)
Para j <- 1 Hasta n Hacer
    v[k] <- matriz[1, j]
    k <- k + 1
FinPara

// 2. Columna Derecha (Baja desde la fila 2, en la ultima
columna)
Para i <- 2 Hasta m Hacer
    v[k] <- matriz[i, n]
    k <- k + 1
FinPara

// 3. Fila Inferior (Retrocede desde la penultima columna)
Si m > 1 Entonces
    Para j <- n - 1 Hasta 1 Con Paso -1 Hacer
        v[k] <- matriz[m, j]
        k <- k + 1
    FinPara
FinSi

// 4. Columna Izquierda (Sube desde la penultima fila
hasta la fila 2)
Si n > 1 Entonces
    Para i <- m - 1 Hasta 2 Con Paso -1 Hacer
        v[k] <- matriz[i, 1]

```

```

        k <- k + 1
    FinPara
FinSi

Escribir "Vector Resultante:"
Para i <- 1 Hasta k - 1 Hacer
    Escribir Sin Saltar v[i], " "
FinPara
FinAlgoritmo

```

Prueba de Escritorio:

Matriz de 3 x 3 con valores del 1 al 9 ordenados.

```

1 2 3
4 5 6
7 8 9

```

Paso	Dirección	Coordenadas procesadas	Valores copiados a v
1	Fila Superior	[1][1], [1][2], [1][3]	1, 2, 3
2	Col. Derecha	[2][3], [3][3]	6, 9
3	Fila Inferior	[3][2], [3][1]	8, 7
4	Col. Izquierda	[2][1]	4
Final			Vector: {1, 2, 3, 6, 9, 8, 7, 4}

Código (C++):

```

#include <iostream>

using namespace std;

```

```

int main() {
    int m, n;
    int matriz[10][10], v[100];

    cout << "Filas (m): "; cin >> m;
    cout << "Columnas (n): "; cin >> n;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cout << "Valor ["<<i<<"] [<<j<<"]: ";
            cin >> matriz[i][j];
        }
    }

    int k = 0; // Indice del vector

    // 1. Superior
    for (int j = 0; j < n; j++) {
        v[k] = matriz[0][j];
        k++;
    }

    // 2. Derecha
    for (int i = 1; i < m; i++) {
        v[k] = matriz[i][n - 1];
        k++;
    }

    // 3. Inferior
    if (m > 1) {
        for (int j = n - 2; j >= 0; j--) {
            v[k] = matriz[m - 1][j];

```

```

        k++;
    }
}

// 4. Izquierda
if (n > 1) {
    for (int i = m - 2; i >= 1; i--) {
        v[k] = matriz[i][0];
        k++;
    }
}

cout << "\nVector de los bordes:\n";
for (int i = 0; i < k; i++) {
    cout << v[i] << " ";
}
cout << endl;

return 0;
}

```

14. Escribir un programa que sume e imprima cada elemento en una matriz de $m \times n$, que forme en ella la letra “N” (como en el bingo)

Pseudocódigo:

Algoritmo LetraNMatriz

Definir n, i, j Como Entero

Definir suma Como Real

Dimension matriz[10, 10]

Escribir "Tamano de la matriz cuadrada (n):"

Leer n

// (Asumimos ingreso de datos aquí)

```

suma <- 0
Escribir "--- Figura N ---"
Para i <- 1 Hasta n Hacer
    Para j <- 1 Hasta n Hacer
        Si j = 1 O i = j O j = n Entonces
            Escribir Sin Saltar matriz[i, j], " "
            suma <- suma + matriz[i, j]
        Sino
            Escribir Sin Saltar " " // Espacio vacio
para dar la forma
    FinSi
    FinPara
    Escribir "" // Salto de linea
FinPara

Escribir "Suma total de la letra N: ", suma
FinAlgoritmo

```

Prueba de Escritorio:

Matriz 3 x 3 rellena con unos (1).

i	j	¿Cumple la condición (j=1 o i=j o j=3)?	Acción
1	1	Sí (j=1, i=j)	Suma e imprime
1	2	No	Imprime espacio en blanco
1	3	Sí (j=3)	Suma e imprime
2	1	Sí (j=1)	Suma e imprime

2	2	Sí (i=j)	Suma e imprime
2	3	Sí (j=3)	Suma e imprime

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    int n;
    float matriz[10][10], suma = 0;

    cout << "Tamano de la matriz (n): "; cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matriz[i][j];
        }
    }

    cout << "\n--- Dibujando la Letra N ---\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Condición lógica de la N: Primera columna OR
            // Diagonal OR Ultima columna
            if (j == 0 || i == j || j == n - 1) {
                cout << matriz[i][j] << "\t";
                suma += matriz[i][j];
            } else {
                cout << " \t"; // Deja un hueco visual
            }
        }
    }
}
```

```

        }
    }
    cout << endl;
}

    cout << "\nLa suma de los elementos que forman la 'N' es:
" << suma << endl;

    return 0;
}

```

(1) Un programa que almacene en una tabla el número de días que tiene cada mes (supondremos que es un año no bisiesto), pida al usuario que le indique un mes (1=enero, 12=diciembre) y muestre en pantalla el número de días que tiene ese mes.

Pseudocódigo:

Algoritmo DiasDelMes

Definir mes Como Entero

Dimension dias[12]

// Inicialización de la tabla

dias[1] <- 31; dias[2] <- 28; dias[3] <- 31; dias[4] <- 30

dias[5] <- 31; dias[6] <- 30; dias[7] <- 31; dias[8] <- 31

dias[9] <- 30; dias[10] <- 31; dias[11] <- 30; dias[12] <-

31

Escribir "Ingrese el numero de mes (1-12):"

Leer mes

Si mes >= 1 Y mes <= 12 Entonces


```
        Escribir "El mes ", mes, " tiene ", dias[mes], "
dias."
```

```
    Sino
```

```
        Escribir "Mes invalido."
```

```
    FinSi
```

```
FinAlgoritmo
```

Prueba de Escritorio:

Caso: mes = 2 (Febrero)

Paso	Variable mes	Condición (1 <= mes <= 12)	Accede a dias[mes]	Salida en pantalla
1	2	Verdadero	dias[2] (en C++ será índice 1)	"El mes 2 tiene 28 días."

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    // Declaración e inicialización directa del vector
    int dias[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
30, 31};
    int mes;

    cout << "Ingrese un mes (1=Enero, 12=Diciembre): ";
    cin >> mes;

    if (mes >= 1 && mes <= 12) {
```

```

        // Restamos 1 porque los vectores en C++ inician en el
indice 0

        cout << "El mes " << mes << " tiene " << dias[mes - 1]
<< " dias.\n";

    } else {

        cout << "Error: Mes invalido.\n";

    }

    return 0;

}

```

(2) Un programa que almacene en una tabla el número de días que tiene cada mes (año no bisiesto), pida al usuario que le indique un mes (ej. 2 para febrero) y un día (ej. el día 15) y diga qué número de día es dentro del año (por ejemplo, el 15 de febrero sería el día número 46, el 31 de diciembre sería el día 365).

Pseudocódigo:

Algoritmo DiaDelAño

Definir mes, día, i, totalDias Como Entero

Dimension dias[12]

// (Asignar los mismos 12 valores a la matriz 'dias')

Escribir "Ingrese el mes (1-12):"

Leer mes

Escribir "Ingrese el día:"

Leer día

totalDias <- 0

Para i <- 1 Hasta mes - 1 Hacer

totalDias <- totalDias + dias[i]

FinPara

```

    totalDias <- totalDias + dia
    Escribir "Es el dia numero ", totalDias, " del ano."
FinAlgoritmo

```

Prueba de Escritorio:

Caso: mes = 3 (Marzo), dia = 15

i	Límite del bucle	dias[i]	totalDias acumulado
Inicial	-	-	0
1	Hasta 2 (mes - 1)	31 (Enero)	0 + 31 = 31
2	Hasta 2	28 (Febrero)	31 + 28 = 59
Final	Suma el día actual	15 (Marzo)	59 + 15 = 74

Código (C++):

```

#include <iostream>
using namespace std;

int main() {
    int dias[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
30, 31};
    int mes, dia, totalDias = 0;

    cout << "Ingrese el mes (1-12): "; cin >> mes;
    cout << "Ingrese el dia: "; cin >> dia;

```

```

// Sumamos los dias de los meses completos anteriores
for (int i = 0; i < mes - 1; i++) {
    totalDias += dias[i];
}

// Sumamos los dias transcurridos del mes actual
totalDias += dia;

cout << "Es el dia numero " << totalDias << " del ano.\n";

return 0;
}

```

(4) Un programa que pida al usuario los datos de una matriz de 3x3, y calcule y muestre su matriz inversa.

Código (C++):

```

#include <iostream>
using namespace std;

int main() {
    // Matriz de 3x6 (3x3 original + 3x3 identidad)
    float M[3][6] = {0};

    cout << "Ingrese la matriz 3x3:\n";
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            cin >> M[i][j];
        }
        M[i][i+3] = 1; // Creando la matriz identidad en el
lado derecho
    }
}

```

```

// Proceso de Gauss-Jordan
for(int i = 0; i < 3; i++) {
    float pivote = M[i][i]; // Diagonal principal

    // Dividimos toda la fila entre el pivote para que la
diagonal sea 1
    for(int j = 0; j < 6; j++) {
        M[i][j] /= pivote;
    }

    // Hacemos ceros en el resto de la columna
    for(int k = 0; k < 3; k++) {
        if(k != i) {
            float factor = M[k][i];
            for(int j = 0; j < 6; j++) {
                M[k][j] -= factor * M[i][j];
            }
        }
    }
}

cout << "\nMatriz Inversa:\n";
for(int i=0; i<3; i++) {
    for(int j=3; j<6; j++) { // Imprimimos solo la mitad
derecha
        cout << M[i][j] << "\t";
    }
    cout << endl;
}
return 0;
}

```

(5) Un programa que pida al usuario los datos de dos matrices de 2x2, y calcule y muestre su producto.

Código (C++):

```
#include <iostream>

using namespace std;

int main() {
    float A[2][2], B[2][2], C[2][2] = {0}; // C se inicializa
    en 0

    cout << "--- Datos Matriz A ---\n";
    for(int i=0; i<2; i++) for(int j=0; j<2; j++) cin >>
    A[i][j];

    cout << "--- Datos Matriz B ---\n";
    for(int i=0; i<2; i++) for(int j=0; j<2; j++) cin >>
    B[i][j];

    // Multiplicación (Algoritmo de 3 bucles)
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++) {
            for(int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    cout << "\nMatriz Producto:\n";
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) cout << C[i][j] << "\t";
        cout << endl;
    }
}
```

```
    }  
    return 0;  
}
```

(6) Un programa que use una matriz de 3x4 para resolver un sistema de 3 ecuaciones con 3 incógnitas usando el método de Gauss (hacer ceros por debajo de la diagonal principal para luego aplicar sustitución regresiva).

Código (C++):

```
#include <iostream>  
using namespace std;  
  
int main() {  
    float M[3][4]; // Matriz de coeficientes y términos  
independientes  
    float x1, x2, x3; // Incógnitas  
  
    cout << "Ingrese la matriz 3x4 (coeficientes y  
resultados):\n";  
    for(int i=0; i<3; i++) {  
        for(int j=0; j<4; j++) {  
            cin >> M[i][j];  
        }  
    }  
  
    // PASO 1: Triangularización (Hacer ceros por debajo de la  
diagonal)  
  
    // Ceros debajo de M[0][0]  
    float factor1 = M[1][0] / M[0][0];  
    for(int j=0; j<4; j++) M[1][j] -= factor1 * M[0][j];
```

```

float factor2 = M[2][0] / M[0][0];
for(int j=0; j<4; j++) M[2][j] -= factor2 * M[0][j];

// Cero debajo de M[1][1]
float factor3 = M[2][1] / M[1][1];
for(int j=0; j<4; j++) M[2][j] -= factor3 * M[1][j];

// PASO 2: Sustitución Regresiva (Despejar desde abajo
hacia arriba)
x3 = M[2][3] / M[2][2];
x2 = (M[1][3] - M[1][2]*x3) / M[1][1];
x1 = (M[0][3] - M[0][1]*x2 - M[0][2]*x3) / M[0][0];

cout << "\nSolucion del sistema:\n";
cout << "x = " << x1 << endl;
cout << "y = " << x2 << endl;
cout << "z = " << x3 << endl;

return 0;
}

```